RESUMEN DE REFERENCIA · .BAT

Todas las sintaxis únicas de un archivo .BAT

Guía categorizada, comentada y con las trampas reales encontradas en los scripts del proyecto.

178 sintaxis explicadas

18 categorías

4 tablas de referencia

26 archivos .bat revisados

- Estructura del script
- Comentarios
- Salida por pantalla — ECHO
- Variables — SET
- Entorno y expansión retardada
- Argumentos y modificadores
- Condicionales — IF
- Etiquetas y saltos — GOTO
- Subrutinas — CALL
- Bucles — FOR
- Redirección y tuberías
- Archivos y carpetas
- Patrones recurrentes
- Errores y trampas reales
- PowerShell desde un .bat
- Comandos del sistema invocados desde el .bat
- Red: comandos de los talleres
- Chuletas de referencia rápida

```
rem --- cabecera típica de cualquier .bat del proyecto ---
@echo off & chcp 65001 >nul
setlocal enabledelayedexpansion
cd /d "%~dp0"
set "REP=%~dp0Reportes" & if not exist "%REP%" mkdir "%REP%"
goto :menu
```

Sistemas Operativos — Segundo corte · Noviembre de 2026

Mapa de la guía

Las categorías siguen el orden en que se necesita cada concepto al escribir un script: primero la estructura, después el control de flujo, luego el procesamiento de datos y al final las recetas y los errores que conviene evitar. Los apéndices reúnen lo que no es sintaxis propia de batch pero se escribe dentro de los .bat.

BASE uso cotidiano · AVANZADO requiere contexto · PELIGRO irreversible o sensible · TRAMPA error frecuente

I FUNDAMENTOS DEL SCRIPT

01 Estructura del script 12
02 Comentarios 3
03 Salida por pantalla — ECHO 6
04 Variables — SET 13
05 Entorno y expansión retardada 9

II ARGUMENTOS Y CONTROL DE FLUJO

06 Argumentos y modificadores 9
07 Condicionales — IF 10
08 Etiquetas y saltos — GOTO 6
09 Subrutinas — CALL 8

III BUCLES, SALIDA Y ARCHIVOS

10 Bucles — FOR 12
11 Redirección y tuberías 11
12 Archivos y carpetas 8

IV RECETAS Y TRAMPAS REALES

13 Patrones recurrentes 13
14 Errores y trampas reales 13

AP APÉNDICES

A PowerShell desde un .bat 7
B Comandos del sistema invocados desde el .bat 26
C Red: comandos de los talleres 12
D Chuletas de referencia rápida 4

PARTE I

Fundamentos del script

Lo que todo .bat necesita para arrancar bien: eco, codificación, variables, expansión y subcadenas.

01	Estructura del script	12
02	Comentarios	3
03	Salida por pantalla — ECHO	6
04	Variables — SET	13
05	Entorno y expansión retardada	9

01 Estructura del script

Las instrucciones de cabecera y de cierre. Definen qué se ve en pantalla, en qué codificación se interpreta el texto y cómo termina el script sin dejar la ventana colgada.

12 SINTAXIS

```
@echo off
```

BASE

Apaga el eco: la consola deja de imprimir cada línea del script y solo muestra lo que imprimes con echo. La @ evita que esa misma línea se muestre. Va siempre en la primera línea útil.

DÓNDE VERLO los 26 .bat

```
chcp 65001 >nul
```

BASE

Cambia la página de códigos de la consola a UTF-8 para que las tildes y la ñ se vean bien en lugar de «Â¡» o «Ã©». El >nul oculta el mensaje «Código de página activo: 65001».

DÓNDE VERLO angela.bat, CRISTIAN.bat, ROYNER.bat

! No basta con escribirlo: guarda el archivo en UTF-8. Programas.bat muestra «Â¡Escaneo» porque su texto y su codificación no coinciden.

```
title Centro de Utilidades
```

BASE

Cambia el título de la ventana de consola. Es cosmético, pero ayuda a identificar el script cuando hay varias ventanas abiertas.

DÓNDE VERLO CAMILO.bat, JORGE.bat, EDWIN2.bat

```
color 0A
```

BASE

Cambia los colores de la consola: el primer dígito es el fondo y el segundo el texto, en hexadecimal (0 negro, 1 azul, A verde claro, F blanco). 0A = fondo negro con texto verde tipo Matrix; 1F = fondo azul y texto blanco. Sin argumentos, color restablece el esquema por defecto.

DÓNDE VERLO CAMILO.bat (0A), DANIEL.bat (1F)

```
cls
```

BASE

Borra la pantalla. Se usa al abrir cada pantalla del menú para que no se acumulen las opciones anteriores.

DÓNDE VERLO JAIRO.bat, JORGE.bat

```
pause
```

BASE

Muestra «Presione una tecla para continuar...» y espera a que el usuario pulse algo. Sirve para que la ventana no se cierre antes de que se lea el resultado.

DÓNDE VERLO casi todos

```
pause >nul
```

BASE

Igual que pause pero sin el mensaje, porque la salida se manda al dispositivo nul, la «papelera» del sistema.

DÓNDE VERLO CRISTIAN.bat, JULIAN.bat

! El dispositivo correcto es nul, nunca null: pause > null crea un archivo llamado «null» en la carpeta y no silencia nada.

```
timeout /t 3 >nul
```

BASE

Espera 3 segundos sin cuenta regresiva y sin pedir tecla. Con /nobreak la espera no se puede cancelar pulsando una tecla. Es la pausa correcta, mejor que el truco ping 127.0.0.1 -n 4 >nul.

DÓNDE VERLO angela.bat, JORGE.bat, ROYNER.bat

```
@pause   ·   @echo off
```

BASE

La @ antepuesta a cualquier comando impide que ese comando se imprima aunque el eco esté activo. Es costumbre de los scripts antiguos.

DÓNDE VERLO EjemplosBatch.bat

```
exit
```

BASE

Cierra la ventana de consola y termina el proceso. Es lo adecuado en scripts que se lanzan con doble clic.

DÓNDE VERLO varios

```
exit /b
```

AVANZADO

Termina el script o la subrutina pero SIN cerrar la ventana: devuelve el control a quien lo llamó. Es lo correcto en menús y en subrutinas invocadas con call.

DÓNDE VERLO CRISTIAN.bat, JAIRO.bat, JULIAN.bat

```
exit /b 0   ·   exit /b 1
```

AVANZADO

Igual que exit /b, pero además devuelve un código de resultado explícito (0 = éxito, 1 = fallo). El llamador lo consulta con if errorlevel.

DÓNDE VERLO JULIAN.bat (:Confirmar)

02 Comentarios

Tres formas de escribir texto que el intérprete ignora. No son intercambiables: una falla dentro de bloques y detrás de tuberías.

3
SINTAXIS

```
rem esto es un comentario
```

BASE

REM = REMark: el resto de la línea se ignora por completo. Es el comentario seguro porque funciona en cualquier posición, incluso dentro de bloques ( ) y detrás de tuberías.

DÓNDE VERLO DANIEL.bat, JEINER.bat, JULIAN.bat

```
@rem comentario
```

BASE

Igual que rem, pero la propia línea del comentario tampoco se muestra si el eco estuviera activo.

DÓNDE VERLO EjemplosBatch.bat

```
:: comentario
```

BASE

Truco habitual: crea una etiqueta vacía y el segundo «:» la deja inalcanzable, así que el intérprete la ignora. Más rápido de escribir que rem.

DÓNDE VERLO EDWIN.bat, JORGE.bat, broma.bat

! No funciona dentro de bloques (...) ni detrás de un pipe: ahí rompe el script. En esos casos usa rem.

03 Salida por pantalla — ECHO

echo escribe texto y expande variables. Sus variantes controlan líneas en blanco, porcentajes y caracteres que el intérprete interpretaría como sintaxis.

6 SINTAXIS

```
echo Hola usuario %USERNAME%
```

BASE

Imprime el texto tal cual, expandiendo antes las variables entre porcentajes: %USERNAME% se sustituye por el usuario de Windows.

DÓNDE VERLO JAIRO.bat, EjemplosBatch.bat

```
echo.
```

BASE

Imprime una línea en blanco; el punto forma parte del comando. Se usa para dar aire entre las secciones de un menú.

DÓNDE VERLO JORGE.bat, JULIAN.bat, ROYNER.bat

! echo sin nada imprime el estado del eco (on/off); echo. siempre imprime una línea vacía.

```
echo Uso de CPU: !CPUL!%%
```

BASE

El signo de porcentaje literal se escribe %%; un % solo intenta abrir una variable y rompe la línea.

DÓNDE VERLO WinSysToolkit-ELKIN.bat

```
echo IP_LOCAL        ^| IP_REMOTA
```

AVANZADO

El circunflejo ^ escapa el carácter siguiente: aquí una barra vertical que NO debe funcionar como tubería. Lo mismo aplica a ^(y ^) para paréntesis literales.

DÓNDE VERLO JORGE.bat, JAIRO.bat

```
echo hola  -^>  listo
```

AVANZADO

El ^ también escapa la flecha -> para que no se confunda con una redirección.

DÓNDE VERLO JAIRO.bat

```
echo ESTE TEXTO >> hola1.txt
```

BASE

echo es la vía normal para escribir o añadir líneas a un archivo, combinado con > o >>. El taller lo usa como forma de «adicionar contenido».

DÓNDE VERLO IRAIDA.bat, EDWIN2.bat

04 Variables — SET

set crea, lee, borra y calcula variables. Aquí está también la lectura del teclado y la sustitución de texto, que es la base de casi todos los menús.

13 SINTAXIS

```
set nombre=Juan
```

BASE

Crea la variable nombre con el valor Juan. No debe haber espacios alrededor del signo igual.

DÓNDE VERLO varios

! set nombre = Juan crea una variable llamada «nombre » (con espacio final) y con valor « Juan»; luego %nombre% no existe. ERIKA.bat tiene exactamente ese bug.

```
set "reporte=%~dp0Reporte.txt"
```

BASE

Forma recomendada: entrecomillar toda la asignación permite valores con espacios y caracteres especiales, sin cortes en el primer espacio ni espacios parásitos.

DÓNDE VERLO CRISTIAN.bat, DANIEL.bat, JULIAN.bat

```
set PASS=admin123
```

PELIGRO

Variable con una clave fija que después se compara en un if para simular un inicio de sesión.

DÓNDE VERLO IRAIDA.bat

! No es protección real: cualquiera que abra el .bat en un editor ve la clave. En batch todo es texto plano.

```
set /p opcion=Seleccione una opcion:
```

BASE

Lee lo que teclea el usuario y lo guarda en opcion, mostrando el prompt sin salto de línea. Es el corazón de todos los menús del proyecto.

DÓNDE VERLO CAMILO.bat, JORGE.bat, YORMAN.bat

```
set /p "op=Elige una opcion: "
```

BASE

Variante segura: entrecomillar el prompt completo evita que espacios accidentales contaminen la variable.

DÓNDE VERLO broma.bat

```
set /p OLDWP=<"estado\fondo_original.txt"
```

AVANZADO

Lee la PRIMERA LÍNEA de un archivo y la guarda en la variable, en lugar de leer del teclado: el < es «entrada desde archivo».

DÓNDE VERLO counter.bat

```
set /a contador=contador+1
```

BASE

El modificador /a convierte set en calculadora: evalúa la parte derecha como expresión aritmética y permite nombrar la variable sin %...% dentro de su propia fórmula. Es el contador típico de los bucles for.

DÓNDE VERLO EjemplosBatch.bat, DANIEL.bat

```
set /a media=(%num1% + %num2%) / 2
```

AVANZADO

Con /a se dispone de paréntesis, + - * / y % (módulo), operadores de bits (& | ^ << >> ~) y las asignaciones += -= *= /=.

DÓNDE VERLO EjemplosBatch.bat

```
set /a "x = x + 1"
```

AVANZADO

Las comillas alrededor de la expresión son opcionales: /a las ignora.

DÓNDE VERLO EjemplosBatch.bat

> ! Dentro de un bloque ( ) o de un for, la variable no se vuelve a leer con %x% hasta la línea siguiente del bloque: para eso existe la expansión retardada.

```
set
```

BASE

Sin argumentos imprime TODAS las variables del entorno activo. Útil para depurar.

DÓNDE VERLO DANIEL.bat: set > variables.txt

```
set raiz
```

BASE

Con solo un nombre o prefijo, set imprime las variables que empiezan por ese texto.

DÓNDE VERLO —

```
set "raiz="
```

BASE

Asignar un valor vacío BORRA la variable. Se hace antes de un set /p para no arrastrar la respuesta anterior si el usuario pulsa Enter sin escribir.

DÓNDE VERLO CRISTIAN.bat, JAIRO.bat, JULIAN.bat

```
set "raiz=%raiz:"=%"
```

AVANZADO

Sustitución de texto dentro de una variable con la forma %var:busca=reemplaza%. Aquí busca todas las comillas y las reemplaza por nada, para quitar las que Windows añade al arrastrar una carpeta sobre la ventana.

DÓNDE VERLO JAIRO.bat

05 Entorno y expansión retardada

Las variables que Windows ya trae, el problema de leer variables dentro de un bloque y las dos formas de resolverlo: setlocal enabledelayedexpansion y if errorlevel.

9 SINTAXIS

```
echo %USERNAME% %COMPUTERNAME% %DATE% %TIME%
```

BASE

Variables predefinidas por Windows: usuario, nombre del equipo, fecha y hora actuales.

DÓNDE VERLO JAIRO.bat, EjemplosBatch.bat

```
%CD%  %HOMEPATH%  %TEMP%  %USERPROFILE%
```

BASE

Carpeta actual, ruta del perfil, carpeta temporal y perfil completo. Otras muy usadas: %SYSTEMROOT%, %ALLUSERSPROFILE%, %NUMBER_OF_PROCESSORS% y %PROCESSOR_ARCHITECTURE%.

DÓNDE VERLO EjemplosBatch.bat, EDWIN2.bat

```
%RANDOM%
```

BASE

Número aleatorio entre 0 y 32767; se usa para nombres únicos y ejercicios de azar.

DÓNDE VERLO EjemplosBatch.bat, taller

```
setlocal enabledelayedexpansion
```

AVANZADO

Activa la expansión retardada: desde aquí las variables se pueden leer con !var! y reflejan los cambios hechos dentro del mismo bloque o del for. Casi todos los scripts serios lo abren y lo cierran con endlocal.

DÓNDE VERLO broma.bat, DANIEL.bat, JAIRO.bat, JULIAN.bat

```
echo !BAR!
```

AVANZADO

Lectura con expansión retardada: !var! devuelve el valor ACTUAL. Con %var% se obtendría el valor que la variable tenía al entrar al bloque.

DÓNDE VERLO EDWIN2.bat (barra de carga)

```
endlocal
```

AVANZADO

Cierra el ámbito abierto por setlocal y descarta las variables creadas dentro, devolviendo el entorno a como estaba.

DÓNDE VERLO EDWIN.bat, JULIAN.bat

```
%errorlevel%   ·   !errorlevel!
```

AVANZADO

Código de resultado del último comando: 0 = todo bien, cualquier otro número = error. Dentro de un bloque la forma %errorlevel% se evalúa tarde; por eso se prefiere !errorlevel! o la forma if errorlevel N.

DÓNDE VERLO EDWIN2.bat, YEISON.bat

```
if errorlevel 1 ( ... ) else ( ... )
```

AVANZADO

Azúcar del intérprete que significa «errorlevel MAYOR O IGUAL que 1». No depende de la expansión, así que funciona igual dentro de bloques.

DÓNDE VERLO YEISON.bat, JULIAN.bat

`for /f ... do set CPUL=%%A → echo !CPUL!%%`

AVANZADO

Patrón clásico: un for /f captura la salida de un comando en una variable y después se imprime con expansión retardada para que no salga vacía.

DÓNDE VERLO WinSysToolkit-ELKIN.bat

PARTE II

Argumentos y control de flujo

Cómo recibe datos el script y cómo decide, salta y reutiliza código.

06 Argumentos y modificadores 9

07 Condicionales — IF 10

08 Etiquetas y saltos — GOTO 6

09 Subrutinas — CALL 8

06 Argumentos y modificadores

Cómo llegan los datos al script desde la línea de comandos o desde un archivo arrastrado sobre el icono, y cómo extraer partes de una ruta.

9 SINTAXIS

```
echo %0 %1 %2 %3
```

BASE

Al ejecutar script.bat Pedro 20: %0 es el nombre del script, %1 es Pedro y %2 es 20. Los argumentos llegan hasta %9.

DÓNDE VERLO EjemplosBatch.bat

```
%*
```

BASE

Contiene TODOS los argumentos a la vez; combinado con un for, se recorren uno a uno.

DÓNDE VERLO for %%n in (%*) do @echo %%n

```
shift
```

AVANZADO

Desplaza los argumentos una posición: %2 pasa a ser %1, %3 a %2, etc. Sirve para procesar una lista mientras queden elementos.

DÓNDE VERLO EjemplosBatch.bat

```
shift /1   ·   shift /2
```

AVANZADO

Descartan a partir del argumento indicado: /1 elimina el primero, /2 elimina el segundo.

DÓNDE VERLO EjemplosBatch.bat

```
%~f1  %~d1  %~p1  %~n1  %~x1
```

AVANZADO

Modificadores que extraen partes del argumento SIN las comillas: ruta completa, unidad, ruta, nombre y extensión.

DÓNDE VERLO EjemplosBatch.bat (programa 6)

```
%~s1  %~a1  %~t1  %~z1
```

AVANZADO

Ruta corta 8.3, atributos del archivo, fecha y hora de modificación, y tamaño en bytes.

DÓNDE VERLO EjemplosBatch.bat

```
cd /d "%~dp0"
```

BASE

%~dp0 es la carpeta donde vive el .bat (unidad + ruta, sin comillas) y cd /d cambia también de unidad. Así las rutas relativas funcionan aunque el script se lance desde otra carpeta.

DÓNDE VERLO broma.bat, snapshot.bat, JULIAN.bat

```
powershell -File "%~dp0comparar.ps1" -Antes "%~1" -Despues "%~2"
```

AVANZADO

Patrón puente: el .bat recibe los argumentos (por ejemplo dos archivos arrastrados sobre el icono) y se los pasa a un script de PowerShell.

DÓNDE VERLO comparar.bat

```
if "%1"=="/?" goto AYUDA
```

AVANZADO

Convención de Windows: si el usuario invoca el script con /? se debe mostrar la ayuda. También se usa if not "%~1"=="hola" echo hola.

DÓNDE VERLO EjemplosBatch.bat

07 Condicionales — IF

Comparar cadenas, números, archivos y códigos de error. Es la sección donde se concentran los errores de sintaxis más comunes del proyecto.

10 SINTAXIS

```
if "%opcion%"=="1" (
    echo Has elegido la opcion 1
    pause
)
```

BASE

Comparación de CADENAS dentro de un bloque: los paréntesis agrupan varias instrucciones bajo una sola condición, y se indentan para leerse. Entrecomilla siempre las dos partes: cuesta nada y evita el fallo más frecuente de batch.

DÓNDE VERLO CAMILO.bat, JAIRO.bat, ROYNER.bat

```
if exist "%~dp0datos.txt" (
    type "%~dp0datos.txt"
) else (
    echo Falta el archivo de datos.
    pause
)
```

BASE

if ... else: el paréntesis de cierre, la palabra else y el de apertura van en la MISMA línea («) else (») con espacios; así se elige entre dos bloques completos, cada uno con sus propias instrucciones.

DÓNDE VERLO DANIEL.bat, EDWIN2.bat, snapshot.bat

```
if "%opcion%"=="1" (
    goto opcion1
) else if "%opcion%"=="2" (
    goto opcion2
) else (
    echo Opcion invalida
    goto menu
)
```

AVANZADO

Cadena de alternativas sin anidar bloques: cada «) else if (...)» añade una condición nueva y el último «) else (» es el caso por defecto. Es el equivalente batch de un switch.

DÓNDE VERLO DANIEL.bat, JEINER.bat, YORMAN.bat

```
if /i "%respuesta%"=="S" (
    echo Actualizando el sistema...
    goto actualizar
)
```

BASE

El modificador /i compara sin distinguir mayúsculas: s, S y sO valen lo mismo. Indispensable en las preguntas de S/N.

DÓNDE VERLO CRISTIAN.bat, ROYNER.bat, JULIAN.bat

```
if not "%CLAVE%"=="%PASS%" (
    echo Clave incorrecta
    goto menu
)
```

BASE

not niega la condición y deja el bloque para el caso contrario; con él se construye el login de IRAIDA.bat.

DÓNDE VERLO IRAIDA.bat

```
if %edad% GEQ 18 (
    echo MAYOR DE EDAD
) else (
    echo MENOR DE EDAD
)
```

BASE

Comparaciones numéricas: EQU igual, NEQ distinto, LSS menor, LEQ menor o igual, GTR mayor, GEQ mayor o igual. Solo para números: para texto se usa ==.

DÓNDE VERLO EjemplosBatch.bat

```
if not exist "%~dp0Reportes\" (
    mkdir "%~dp0Reportes"
    echo Carpeta creada.
) else (
    echo La carpeta ya existe.
)
```

BASE

La barra invertida antes de la comilla de cierre hace que if exist compruebe una CARPETA y no un archivo; con if not exist se crea solo si falta, el patrón de las carpetas de salida.

DÓNDE VERLO WinSysToolkit-ELKIN.bat, DANIEL.bat, JULIAN.bat

```
net session >nul 2>&1
if errorlevel 1 (
    echo Necesitas permisos de administrador.
    pause
    exit /b 1
)
```

AVANZADO

if errorlevel N significa «errorlevel mayor o igual que N»: detecta cualquier error sin depender de la expansión y por eso es la forma preferible dentro de bloques. Aquí comprueba si el script corre como administrador.

DÓNDE VERLO JORGE.bat, EDWIN.bat, YORMAN.bat

```
if defined VAR (
    echo VAR vale %VAR%
) else (
    echo VAR no esta definida
)
```

AVANZADO

if defined comprueba si la variable EXISTE, aunque esté vacía, sin expandirla: la forma segura de preguntar por una variable que puede no haberse creado nunca.

DÓNDE VERLO no aparece en los scripts; variante de apoyo

```
if %opcion%==1 (
    echo Opcion 1
)
```

TRAMPA

TRAMPA: sin comillas, si la variable quedó vacía la línea se convierte en «if ==1» y el script muere con error de sintaxis. Pasa siempre que el usuario pulsa Enter en un set /p.

DÓNDE VERLO Correcto: if "%opcion%"=="1" (...)

08 Etiquetas y saltos — GOTO

Batch no tiene control de flujo estructurado: tiene etiquetas y saltos. Con eso se construyen los menús, las páginas de ayuda y hasta un while simulado.

6 SINTAXIS

```
:menu
```

BASE

Crea una etiqueta: es solo un marcador, no ejecuta nada. El nombre no distingue mayúsculas, así que :menu y :MENU son la misma.

DÓNDE VERLO CAMILO.bat, JORGE.bat

```
goto menu
```

BASE

Salta a la etiqueta. También se escribe goto :menu con dos puntos y es equivalente. Es el mecanismo de todos los menús: cada opción termina volviendo a :menu para repintarlo.

DÓNDE VERLO los 20+ scripts del proyecto

```
goto :EOF
```

AVANZADO

Sintaxis reservada que significa «fin del script o de la subrutina actual». Es el return genérico: detiene la ejecución sin cerrar la ventana.

DÓNDE VERLO DANIEL.bat, EDWIN.bat, JAIRO.bat

```
:AYUDA ... :FIN   +   goto AYUDA
```

AVANZADO

Patrón de «páginas» del script: cada etiqueta es una pantalla y goto salta entre ellas.

DÓNDE VERLO EjemplosBatch.bat (fragmento repite)

```
:while1 ... if %x% leq 8 ( ... ) ... goto :while1
```

AVANZADO

WHILE simulado: batch no tiene while, se inventa con una etiqueta, un if y un goto que vuelve a la etiqueta. WinSysToolkit lo usa para el monitoreo continuo (timeout /t 3 + goto LOOP_MON).

DÓNDE VERLO EjemplosBatch.bat, WinSysToolkit-ELKIN.bat

```
:estudio ... :descanso ... (sin goto final)
```

TRAMPA

Si una sección no termina en goto menu o goto fin, la ejecución SIGUE y cae en la etiqueta siguiente, que se ejecuta sin que el usuario la elija.

DÓNDE VERLO angela.bat

> ! Cierra siempre cada sección con un salto explícito: pause + goto menu, o goto :EOF.

09 Subrutinas — CALL

call convierte una etiqueta en una función reutilizable con parámetros y valor de retorno. Es la diferencia entre un script copiado y pegado y uno mantenible.

8 SINTAXIS

```
call :suma 5 3
```

BASE

Ejecuta la etiqueta :suma del propio archivo pasándole argumentos (dentro son %1 y %2) y regresa a la línea siguiente cuando la subrutina hace goto :EOF o exit /b.

DÓNDE VERLO EjemplosBatch.bat (:suma, :resta)

```
call :MoverPorTipo "01_Gestion" "docx doc pdf"
```

AVANZADO

Subrutina con varios parámetros: dentro, %~1 y %~2 los reciben sin comillas. Evita repetir cinco veces el mismo bucle de mover archivos por extensión.

DÓNDE VERLO JAIRO.bat

```
call :Confirmar "¿Continuar con el analisis?"
```

AVANZADO

Subrutina reutilizable que pregunta S/N y devuelve el resultado como código: exit /b 0 si acepta y exit /b 1 si no. El llamador hace if errorlevel 1 goto Menu.

DÓNDE VERLO JULIAN.bat

```
call :REVISAR_SERVICIO "MapsBroker" "Descripcion" "Manual"
```

AVANZADO

Lo más avanzado: una subrutina recibe servicio, justificación y sugerencia, usa setlocal/endlocal para no ensuciar el entorno y se llama una vez por cada servicio.

DÓNDE VERLO EDWIN.bat

```
call "%~dp0counter.bat"
```

AVANZADO

call también lanza OTRO archivo .bat sin cerrar la ventana actual. Sin call, el script externo toma el control y el actual no continúa.

DÓNDE VERLO broma.bat

```
call npm install -g pnpm
```

AVANZADO

call envuelve también comandos que en realidad son .bat o .cmd (npm lo es); sin el call, el script actual terminaría ahí.

DÓNDE VERLO YORMAN.bat

```
@echo off&call:main&goto:EOF
```

AVANZADO

Todo en una línea con & como separador de comandos: apaga el eco, llama a la subrutina principal y termina. Nota: call:main sin espacio también es válido.

DÓNDE VERLO EjemplosBatch.bat

! El goto:EOF final es obligatorio: sin él, al terminar :main la ejecución «cae» dentro de :suma.

```
comando1 & comando2
```

BASE

El ampersand encadena varias instrucciones en la misma línea; se ejecutan una tras otra sin importar el resultado de la anterior.

DÓNDE VERLO EjemplosBatch.bat, IRAIDA.bat

PARTE III

Bucles, salida y archivos

Recorrer, filtrar, redirigir y trabajar con archivos y carpetas.

10	Bucles — FOR	12
11	Redirección y tuberías	11
12	Archivos y carpetas	8

10 Bucles — FOR

El comando más potente del batch: recorre listas, archivos, carpetas, árboles completos y la salida de otros comandos. Sus opciones /d, /R y /f son las que más se usan en el proyecto.


```
for %%x in (1 3 4 6 87) do (
    echo Numero: %%x
)
```

BASE

Recorre una lista literal metiendo varias instrucciones dentro del bloque. En un .bat la variable del bucle lleva DOS porcentajes: con uno solo se expandiría como variable. La letra es libre (i, f, A, E...).

DÓNDE VERLO EjemplosBatch.bat

```
for %%f in (*.txt) do (
    echo Procesando %%f
    type "%%f" >> "salida.txt"
)
```

BASE

La lista puede ser un comodín: aquí todos los .txt de la carpeta, y el bloque procesa cada archivo. Con (*) son todos los archivos de la carpeta actual.

DÓNDE VERLO EjemplosBatch.bat, YEISON.bat

```
for /l %%i in (1,1,5) do (
    echo Iteracion %%i
)
```

BASE

/l genera la lista sola: inicio, paso y final (1,1,5 produce 1 2 3 4 5). Sustituye a escribir los números a mano y es la forma limpia de contar.

DÓNDE VERLO no aparece en los scripts; variante de apoyo

```
for /d %%x in ("%temp%\*") do (
    echo Borrando %%x
    rd /s /q "%%x"
)
```

AVANZADO

/d hace que el bucle recorra solo CARPETAS del comodín, sin entrar en archivos; aquí borra cada carpeta temporal.

DÓNDE VERLO CAMILO.bat

```
for /r "%~dp0" %%E in (*.log) do (
    echo Encontrado: %%E
)
```

AVANZADO

/R recorre RECURSIVAMENTE la carpeta indicada y todas sus subcarpetas buscando el patrón; si se omite la ruta, empieza en la carpeta actual.

DÓNDE VERLO EjemplosBatch.bat

```
for /f "tokens=5" %%p in ('netstat -ano ^| find "ESTABLISHED"') do (
    echo PID conectado: %%p
)
```

AVANZADO

for /f analiza TEXTO línea a línea; lo encerrado en comillas simples es un COMANDO cuya salida se procesa. tokens=5 toma la quinta palabra de cada línea. El ^| es vital: escapa la tubería para que la consuma el comando interno.

DÓNDE VERLO JORGE.bat, DANIEL.bat

```
for /f "tokens=1,2 delims=," %%a in (datos.csv) do (
    echo Usuario: %%a - Clave: %%b
)
```

AVANZADO

tokens=1,2 asigna la primera palabra a %%a y la segunda a %%b (y sigue el alfabeto: c, d…). delims=, cambia el separador: coma en lugar de espacio y tabulador.

DÓNDE VERLO DANIEL.bat, JULIAN.bat

```
for /f "skip=3 tokens=1,2" %%a in ('tasklist') do (
    echo Proceso: %%a con %%b
)
```

AVANZADO

skip=N descarta las N primeras líneas, típicamente los encabezados de la tabla, y tokens=1,2 se queda con el nombre del proceso y su consumo de memoria.

DÓNDE VERLO DANIEL.bat

```
for /f "tokens=2 delims=:" %%i in ('ipconfig ^| findstr /i "IPv4"') do (
    echo IP: %%i
)
```

AVANZADO

Con delims=: cada segmento entre dos puntos es un token, así que el valor queda en el segundo; sirve para limpiar la salida de ipconfig. La variante tokens=2,* dejaría el resto de la línea en %%j.

DÓNDE VERLO EjemplosBatch.bat

```
for %%E in (txt log) do (
    for %%F in ("*.%%E") do (
        echo Archivo: %%~nxF
    )
)
```

AVANZADO

FOR ANIDADOS con los paréntesis balanceados. Dentro se usan los modificadores de la variable de bucle: %%~nF (nombre sin extensión), %%~xF (solo extensión), %%~nxF (nombre completo), %%~dF y %%~pF (unidad y ruta).

DÓNDE VERLO JAIRO.bat

```
set "BAR="
for %%A in (1 2 3 4 5) do (
    set "BAR=!BAR!#"
    cls
    echo !BAR!
)
```

AVANZADO

Dentro de un bloque de for, toda variable que se MODIFICA debe leerse con !…!, porque %…% se congela al entrar al bloque. Así se arma la barra de carga de EDWIN2.bat.

DÓNDE VERLO EDWIN2.bat, JAIRO.bat

```
for %%i in (1 2 3) do (
    call :RUN_OPTION %%i
)
```

AVANZADO

Patrón «ejecutar todo»: el for llama a la subrutina pasándole el número de opción, para la opción A de ejecutar todas las auditorías.

DÓNDE VERLO DANIEL.bat

11 Redirección y tuberías

Mover la salida y los errores entre la pantalla, los archivos y otros comandos. Es la base de todos los reportes y logs del proyecto.


```
dir > todos.txt
```

BASE

> envía la salida a un archivo SOBRESCRIBIENDO lo que hubiera dentro.

DÓNDE VERLO EjemplosBatch.bat

```
ipconfig >> archivo.txt
```

BASE

>> AÑADE al final del archivo sin borrar lo anterior. Es el más usado en reportes e historiales.

DÓNDE VERLO JHOJAN.bat, EDWIN2.bat, DANIEL.bat

```
systeminfo > "%REPORTS%\systeminfo.txt" 2>nul
```

BASE

2>nul oculta los ERRORES (stderr) mientras la salida normal sí va al archivo. Si el comando falla por permisos, no ensucia la consola.

DÓNDE VERLO DANIEL.bat, EDWIN.bat, JULIAN.bat

```
net session >nul 2>&1
```

BASE

El combo para callar un comando por completo: >nul manda la salida normal a la basura y 2>&1 redirige los errores al mismo sitio. No se ve absolutamente nada.

DÓNDE VERLO JORGE.bat, JUANDIEGO-AUTO.bat, JULIAN.bat

```
ipconfig /all | findstr /i "IPv4"
```

BASE

La tubería | usa la salida de un comando como ENTRADA del siguiente, que la filtra.

DÓNDE VERLO EDWIN2.bat, WinSysToolkit-ELKIN.bat

```
findstr /i /c:"texto" /b /v
```

AVANZADO

Filtro potente: /i sin distinguir mayúsculas, /c busca la frase completa, /b ancla al inicio de línea y /v invierte el resultado (muestra lo que NO coincide).

DÓNDE VERLO EDWIN2.bat, JULIAN.bat, DANIEL.bat

```
( echo Fecha: %date% & winget list ) > "%reporte%" 2>&1
```

AVANZADO

Los PARÉNTESIS agrupan varias instrucciones como una sola y la redirección se aplica al conjunto: todo el bloque cae al archivo. Es la forma limpia de generar reportes multi-línea.

DÓNDE VERLO CRISTIAN.bat, ROYNER.bat, YORMAN.bat

```
echo [%date% %time%] Opcion 1>>historial.log
```

BASE

Patrón de LOG: una línea por acción, con marca de tiempo y acumulada con >>.

DÓNDE VERLO IRAIDA.bat, EDWIN2.bat, DANIEL.bat

! Ojo al espacio antes de >>: «echo algo >> archivo» guarda «algo » con espacio final; si molesta, escribe «echo algo>> archivo».

```
type historial.log
```

BASE

Imprime el contenido de un archivo en pantalla (el equivalente a cat de Linux).

DÓNDE VERLO IRAIDA.bat

```
del /q /f "%TEMP%\*" >nul 2>&1
```

PELIGRO

del borra archivos: /q no pregunta uno por uno y /f fuerza los de solo lectura; el >nul 2>&1 silencia los que estén en uso.

DÓNDE VERLO JORGE.bat, JULIAN.bat, CAMILO.bat

```
rd /s /q "C:\ruta"
```

PELIGRO

rd borra una carpeta CON todo su contenido, sin preguntar (/q) y con subcarpetas (/s).

DÓNDE VERLO JORGE.bat, CAMILO.bat

! Es irreversible: no pasa por la papelera. Confirma siempre con una pregunta S/N antes de usarlo.

12 Archivos y carpetas

Los comandos de manipulación de archivos que aparecen dentro de los scripts: listar, crear, copiar, comparar y dibujar árboles.

8 SINTAXIS

```
dir /a-d /b /on   ·   dir /ad /b /on
```

BASE

dir lista un directorio: /a-d solo archivos (sin carpetas), /ad solo carpetas, /b formato limpio sin adornos y /on ordena por nombre.

DÓNDE VERLO EjemplosBatch.bat (list.bat)

```
mkdir carpeta   ·   md carpeta
```

BASE

Crean una carpeta (md y mkdir son lo mismo). Si ya existe dan un error inofensivo, por eso se usan junto a if not exist.

DÓNDE VERLO DANIEL.bat, YEISON.bat

```
copy /y origen destino   ·   move origen destino
```

BASE

copy copia (con /y no pregunta antes de sobrescribir) y move mueve o renombra.

DÓNDE VERLO YEISON.bat, JAIRO.bat

```
xcopy "%origen%" "%destino%" /E /I
```

BASE

Copia árboles completos: /E incluye carpetas vacías y /I asume que el destino es una carpeta aunque esté vacío.

DÓNDE VERLO ERIKA.bat

```
del Historial\*.tmp
```

BASE

Borra por comodín los temporales creados, por ejemplo después de comparar instantáneas.

DÓNDE VERLO YEISON.bat

```
attrib "archivo"
```

BASE

Muestra o cambia los atributos de un archivo: oculto, solo lectura, sistema, archivo.

DÓNDE VERLO varios

```
fc archivo1 archivo2
```

BASE

File Compare: compara dos archivos y devuelve errorlevel distinto de 0 si difieren. Con /A muestra solo las líneas que cambian, /N añade el número de línea y /B compara en binario.

DÓNDE VERLO YEISON.bat

```
tree C:\ /f /a
```

BASE

Dibuja el árbol de carpetas: /f incluye los archivos y /a usa solo caracteres ASCII, útil para guardarlo en un reporte.

DÓNDE VERLO DANIEL.bat

PARTE IV

Recetas y trampas reales

Los patrones que se repiten en todos los scripts y los errores que hay que evitar.

13 **Patrones recurrentes** 13

14 **Errores y trampas reales** 13

13 Patrones recurrentes

Combinaciones completas que aparecen una y otra vez en los 26 scripts. Más que sintaxis, son las recetas con las que se arma cualquier .bat de administración.


```
:menu → cls → echo opciones → set /p → if → goto
```

BASE

EL patrón número uno del taller: pantalla limpia, opciones numeradas, lectura con set /p, un if por opción que salta a su etiqueta y cada opción termina en pause + goto menu.

DÓNDE VERLO CAMILO.bat, EDWIN.bat, JORGE.bat, IRAIDA.bat

```
if "%opcion%"=="" goto menu
```

BASE

Validación de entrada vacía o inválida antes de procesarla; la versión rápida es dejar caer al final del menú con un goto menu sin condición, como opción por defecto.

DÓNDE VERLO CRISTIAN.bat, JAIRO.bat

```
set "op="   (antes de set /p)
```

BASE

Higiene de menús: borrar la variable antes de leerla evita que, al pulsar Enter sin escribir, quede el valor anterior y se repita la acción por accidente.

DÓNDE VERLO CRISTIAN.bat, JAIRO.bat, broma.bat

```
set /p resp= Desea continuar? (S/N):   →   if /i "%resp%"=="S"
```

BASE

Confirmación antes de acciones destructivas o largas, presente en actualizaciones, borrados y análisis del sistema.

DÓNDE VERLO CRISTIAN.bat, ROYNER.bat, JULIAN.bat

```
net session >nul 2>&1   &   if %errorlevel% neq 0 ( ... )
```

AVANZADO

Comprobación de privilegios: si el script no corre como administrador, se avisa o se relanza elevado.

DÓNDE VERLO JORGE.bat, JUANDIEGO-AUTO.bat, JULIAN.bat

```
set "salida=%~dp0Reportes" & if not exist "%salida%" mkdir "%salida%"
```

BASE

Carpeta de salida junto al .bat, creada solo si falta, para que los reportes no se dispersen por el disco.

DÓNDE VERLO DANIEL.bat, EDWIN.bat, JULIAN.bat

```
( echo ... & comando & echo ... ) > "%archivo%"
```

BASE

Reporte de una sola pasada: agrupar todo el bloque y redirigir el conjunto. Cambia el orden y el contenido según el autor, pero la idea es idéntica.

DÓNDE VERLO CRISTIAN.bat, ROYNER.bat, YORMAN.bat

```
echo [%date% %time%] mensaje>>"%LOG%"
```

BASE

Registro de actividad: una línea con marca de tiempo por cada acción ejecutada.

DÓNDE VERLO EDWIN2.bat, IRAIDA.bat, DANIEL.bat

```
set "TS=%DATE:/=-%_%TIME::=-%"
```

BASE

Marca de tiempo para nombres de archivo, en su versión 100 % batch (la variante con Get-Date es la inmune a la configuración regional).

DÓNDE VERLO ROYNER.bat, snapshot.bat

```
set BAR=   →   for %%A in (1 2 3 ...) do ( set BAR=!BAR!# & cls & echo !BAR! )
```

AVANZADO

Barra de carga o animación de presentación: la variable crece dentro del for y se repinta con cls. El «ping 127.0.0.1 -n 4 >nul» que suele acompañarla es un sleep casero; timeout es mejor.

DÓNDE VERLO EDWIN2.bat

```
if exist "estado\fondo_original.txt" ( ... ) else ( ... )
```

AVANZADO

Archivos de estado (.flag): el script guarda en una carpeta «estado» qué se aplicó, para que otro script sepa qué deshacer. Es la base de cualquier script reversible.

DÓNDE VERLO broma.bat / counter.bat

```
fc viejo nuevo >nul   →   if errorlevel 1 ( avisar )
```

AVANZADO

Detección de cambios: se comparan dos instantáneas (procesos, usuarios, drivers) y se avisa si difieren.

DÓNDE VERLO YEISON.bat

```
cd /d "%~dp0"   +   set "OUT=%~dp0snapshots\SNAP_%TS%.txt"
```

BASE

Rutas seguras: anclarse a la carpeta del script y construir las rutas relativas desde ahí, siempre entre comillas.

DÓNDE VERLO broma.bat, snapshot.bat, JULIAN.bat

14 Errores y trampas reales

Fallos que ya existen en los scripts de referencia y que conviene reconocer al leerlos. Cada tarjeta muestra el código tal como está escrito (mal) y la corrección.

```
set origen = C:\Users\...\SO Sem 1
```

TRAMPA

ERROR (ERIKA.bat). Los espacios alrededor del = crean la variable «origen » con un espacio de valor delante; luego %origen% se expande vacío y el xcopy no copia nada.

DÓNDE VERLO Correcto: set "origen=C:\Users\...\SO Sem 1"

```
if /i "%confirmar% == "S" (
```

TRAMPA

ERROR (ERIKA.bat). Las comillas están fuera de lugar: la comparación nunca se cumple bien y además rompe el análisis de la línea.

DÓNDE VERLO Correcto: if /i "%confirmar%"=="S" (

```
if exist "%destino_base%\%carperta_destino%"(
```

TRAMPA

ERROR (ERIKA.bat). Faltan los espacios antes del paréntesis y la variable está mal escrita (arriba se definió carpeta_destino). Batch no avisa del typo: expande vacío.

DÓNDE VERLO Correcto: if exist "%ruta%" (...)

```
) else(
```

TRAMPA

ERROR (ERIKA.bat). El else necesita espacios: escrito así, el intérprete lo toma como parte del paréntesis y el script muere.

DÓNDE VERLO Correcto:) else (

```
pause > null
```

TRAMPA

ERROR (varios fragmentos de EjemplosBatch.bat). «null» no existe en Windows: crea un ARCHIVO llamado null en la carpeta y la pausa no se silencia.

DÓNDE VERLO Correcto: pause >nul

```
CD..   ·   CD C:Windowssystem32
```

TRAMPA

ERROR (EjemplosBatch.bat). CD.. sin espacio no funciona (debe ser cd ..) y «C:Windowssystem32» no es una ruta válida porque le faltan las barras.

DÓNDE VERLO Correcto: cd .. · cd C:\Windows\system32

```
if exist %command% echo Compiling... [salto] else goto :error
```

TRAMPA

ERROR (EjemplosBatch.bat, bloque Java). La variable sin comillas revienta si está vacía, y el else en línea aparte no funciona tras una instrucción simple.

DÓNDE VERLO Correcto: if exist "%command%" (...) else (...)

```
set /p opcion=%1   (en un menú)
```

TRAMPA

RARO (EjemplosBatch.bat, menu1 a menu10). %1 es el primer argumento del script; si no se le pasa, el prompt queda vacío y el usuario no sabe qué teclear.

DÓNDE VERLO Correcto: set /p opcion=Digite el numero:

```
if %opcion%==1 ...   (con opcion vacía)
```

TRAMPA

TRAMPA GENERAL: cualquier if %var%==x se rompe cuando la variable quedó vacía, porque la línea se convierte en «if ==1».

DÓNDE VERLO Correcto: if "%opcion%"=="1"

```
wmic /namespace:\\root\SecurityCenter2 ...
```

TRAMPA

OJO (ROYNER.bat): una barra invertida antes de la comilla de cierre no escapa nada dentro de comillas dobles. Una ruta que termina en \ dentro de comillas «rompe» la comilla final.

DÓNDE VERLO Correcto: "%ruta%\\" o armar la ruta sin barra final

```
echo Escaneo completado   (acentos sin chcp)
```

TRAMPA

ERROR FRECUENTE: ver «Â¡» o «Ã©» significa que el archivo está en UTF-8 pero la consola usa otra codificación (o al revés).

DÓNDE VERLO Correcto: chcp 65001 >nul en la primera línea + archivo guardado en UTF-8

```
for ... do (  :: comentario  )
```

TRAMPA

TRAMPA: los comentarios con :: fallan dentro de bloques parentizados y detrás de tuberías.

DÓNDE VERLO Correcto: usar rem en esos casos

```
echo Progreso: 50%   (porcentaje suelto)
```

TRAMPA

TRAMPA: un % solo intenta abrir una variable y puede comerse el resto de la línea o imprimir algo inesperado.

DÓNDE VERLO Correcto: echo Progreso: 50%%

PARTE AP

Apéndices

PowerShell desde batch, comandos del sistema y de red, y chuletas de referencia rápida.

A	PowerShell desde un .bat	7
B	Comandos del sistema invocados desde el .bat	26
C	Red: comandos de los talleres	12
D	Chuletas de referencia rápida	4

A PowerShell desde un .bat

Cuando batch no da abasto (hardware, WMI, listar archivos grandes, elevar permisos), la solución del proyecto es delegar en PowerShell desde la misma ventana.

7 SINTAXIS

```
powershell -NoProfile -Command "Get-ChildItem C:\ -File -Recurse | Sort-Object Length -Descending | Select-Object -First 20"
```

AVANZADO

Lanza PowerShell para una instrucción concreta. -NoProfile evita cargar tu configuración (más rápido y predecible).

DÓNDE VERLO EspacioDisco.bat, Programas.bat

```
powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0recolectar.ps1" -OutFile "%OUT%"
```

AVANZADO

-ExecutionPolicy Bypass esquiva la política de ejecución y -File ejecuta un script completo con parámetros. Es el patrón recomendado cuando la lógica es seria: el .bat maneja menús y rutas, el .ps1 hace el trabajo.

DÓNDE VERLO snapshot.bat, comparar.bat, enviar_correo.bat

```
powershell -Command "(New-Object -ComObject SAPI.SpVoice).Speak('Hola')"
```

PELIGRO

PowerShell puede usar objetos COM de Windows; aquí el sintetizador de voz. También se usa para SendKeys y MessageBox.

DÓNDE VERLO broma.bat

```
powershell -Command "Start-Process '%~f0' -Verb RunAs"
```

AVANZADO

AUTOELEVACIÓN: relanza el propio .bat (%~f0 es su ruta completa) pidiendo permisos de administrador. Es el equivalente moderno de runas.

DÓNDE VERLO JUANDIEGO-AUTO.bat, JULIAN.bat

! Al elevar, Windows arranca el script desde System32: añade cd /d "%~dp0" al principio, como hace JULIAN.bat.

```
Get-CimInstance -ClassName Win32_PhysicalMemory | Select-Object BankLabel, Capacity, Speed
```

AVANZADO

El reemplazo moderno de wmic memorychip. Microsoft desaconseja WMIC y pide migrar a Get-CimInstance.

DÓNDE VERLO JHOJAN.bat, EDWIN2.bat

```
(Get-Process -Id 20640).Threads   ·   Get-Process | ForEach-Object { $_.Threads }
```

AVANZADO

Lista los HILOS de un proceso concreto. Sirve para conectar la teoría de procesos e hilos con la práctica: el proceso 20640 tenía 43 hilos.

DÓNDE VERLO Taller baterías 1

```
... | Select-Object Id | Export-Csv -Path C:\Hilos.csv -NoTypeInformation
```

AVANZADO

Patrón de reporte: exportar el resultado a CSV sin cabecera de tipo, listo para abrir en Excel.

DÓNDE VERLO Taller baterías 1

B Comandos del sistema invocados desde el .bat

26
SINTAXIS

No son sintaxis de batch, pero se escriben dentro de los scripts y son los que hacen el trabajo: procesos, servicios, red, energía, reparación y paquetes.

```
start notepad   ·   start "" notepad
```

BASE

start abre un programa en una ventana NUEVA sin bloquear el script. Las comillas vacías son el TÍTULO de la ventana: son necesarias si la ruta del programa tiene espacios, porque si no start las interpretaría como título.

DÓNDE VERLO EDWIN2.bat

```
start "" "https://github.com"
```

BASE

start también abre una URL con el navegador predeterminado.

DÓNDE VERLO JORGE.bat, taller

```
start cmd /k systeminfo
```

BASE

cmd /k abre una consola y deja el comando corriendo sin cerrarse; con /c la cerraría al terminar. Patrón para lanzar herramientas del sistema en ventanas aparte.

DÓNDE VERLO EjemplosBatch.bat

```
start /b cmd /c java -version
```

AVANZADO

/b hace que el comando corra en la MISMA ventana, sin abrir otra.

DÓNDE VERLO EjemplosBatch.bat

```
choice /c 1234567890A /n /m "Seleccione una opcion: "
```

AVANZADO

Alternativa moderna a set /p para menús: no deja teclear texto, solo pulsar una de las teclas listadas, y el código de la tecla queda en %errorlevel% (1 = primera tecla, 2 = segunda…). /n oculta la lista y /m define el mensaje.

DÓNDE VERLO DANIEL.bat

```
tasklist /v   ·   taskkill /PID 1234 /F   ·   taskkill /f /im explorer.exe
```

PELIGRO

Lista procesos (tasklist /v con detalle) y los mata por PID o por nombre de imagen (/im), con /f de fuerza.

DÓNDE VERLO CAMILO.bat, broma.bat, counter.bat

```
systeminfo   ·   hostname   ·   whoami   ·   ver
```

BASE

Información básica del equipo y de la sesión: inventario completo, nombre del equipo, usuario actual y versión corta de Windows.

DÓNDE VERLO DANIEL.bat, ROYNER.bat

```
ipconfig /all   ·   /flushdns   ·   /release   ·   /renew   ·   /displaydns
```

BASE

Configuración de red completa (MAC, DHCP, DNS), vaciar la caché DNS, liberar y renovar la IP, y ver la caché de resoluciones.

DÓNDE VERLO JORGE.bat, JULIAN.bat

```
ping google.com -n 4   ·   ping -t
```

BASE

Prueba de conectividad. En Windows -n 4 envía cuatro paquetes (por defecto ya son cuatro; en Linux el número va con -c) y -t repite sin parar hasta Ctrl+C.

DÓNDE VERLO EDWIN2.bat

```
netstat -ano
```

BASE

Conexiones de red con el PID del proceso dueño: -a todas, -n en números (sin resolver DNS) y -o el PID. Se filtra con find ESTABLISHED o findstr LISTENING.

DÓNDE VERLO JORGE.bat, DANIEL.bat

```
net user   ·   net localgroup   ·   net start
```

AVANZADO

Gestión de usuarios locales, grupos y servicios iniciados. Con net user nombre clave /add se crea un usuario y con /delete se borra.

DÓNDE VERLO WinSysToolkit-ELKIN.bat

```
sc query type= service state= all
```

AVANZADO

Consulta los servicios de Windows. sc exige un ESPACIO antes de los signos igual: «state= all», nunca «state=all».

DÓNDE VERLO WinSysToolkit-ELKIN.bat, DANIEL.bat

```
wmic logicaldisk get name, freespace, size   ·   wmic diskdrive get model, size, status
```

AVANZADO

Espacio libre y total por unidad lógica, e información física de los discos con su estado. Con «wmic path SoftwareLicensingService get OA3xOriginalProductKey» se extrae además la licencia OEM de Windows.

DÓNDE VERLO IRAIDA.bat, ROYNER.bat, WinSysToolkit-ELKIN.bat

```
wmic memorychip get BankLabel, Capacity, Speed
```

AVANZADO

WMIC consulta hardware y sistema por consola: RAM, discos, CPU, placa base, licencia. Está obsoleto en Windows 11 y Microsoft pide migrar a PowerShell/Get-CimInstance.

DÓNDE VERLO JUANDIEGO-AUTO.bat, ROYNER.bat

```
winget install --id Git.Git --silent   ·   winget upgrade --all
```

BASE

El gestor de paquetes de Windows: instala y actualiza programas sin descargar instaladores a mano. winget list --upgrade-available lista lo actualizable.

DÓNDE VERLO YORMAN.bat, CRISTIAN.bat, JEINER.bat

```
where git >nul 2>&1
```

BASE

Busca un ejecutable en el PATH (como which en Linux). Con if %errorlevel%==0 sirve para verificar si algo está instalado.

DÓNDE VERLO YORMAN.bat

```
shutdown.exe -s -t 90 -c "Saludos chao"
```

PELIGRO

Apaga el equipo en 90 segundos con un mensaje: -s apagar (-r reiniciar), -t segundos de espera, -c comentario en pantalla.

DÓNDE VERLO EjemplosBatch.bat, taller

```
shutdown -a
```

BASE

ABORTA un apagado programado. Es el salvavidas si el comando anterior se ejecutó por error.

DÓNDE VERLO taller baterías 1

```
powercfg /batteryreport /output "bateria.html"
```

BASE

Genera el informe de batería en HTML: ciclos de carga y comparación entre capacidad de diseño y capacidad real de carga.

DÓNDE VERLO CAMILO.bat, JULIAN.bat

```
powercfg /energy   ·   powercfg /list
```

BASE

/energy analiza 60 segundos y reporta problemas de eficiencia energética; /list muestra los esquemas de energía disponibles.

DÓNDE VERLO JULIAN.bat, DANIEL.bat

```
powercfg -attributes 2a737441-... 48352abd-... -ATTRIB_HIDE
```

PELIGRO

Muestra u oculta una opción avanzada de energía que el panel no expone, cambiando sus atributos con los GUID de subgrupo y de opción. Antes se hacía a mano en REGEDIT (valor 2 en ATRIBUTES).

DÓNDE VERLO taller baterías 1

```
sfc /scannow   ·   DISM /Online /Cleanup-Image /RestoreHealth   ·   chkdsk C:
/scan
```

AVANZADO

Reparación del sistema: sfc repara archivos del sistema, DISM repara la imagen de la que se abastece sfc (por eso va primero) y chkdsk /scan revisa el disco en caliente sin reiniciar.

DÓNDE VERLO JORGE.bat, JULIAN.bat, EDWIN2.bat

```
reg add "HKCU\...\Advanced" /v HideIcons /t REG_DWORD /d 1 /f
```

PELIGRO

Modifica el registro de Windows: /v nombre del valor, /t tipo, /d dato y /f fuerza sin preguntar. reg query lee valores.

DÓNDE VERLO broma.bat, counter.bat

```
driverquery /v   ·   cmdkey /list   ·   runas /user:sistemas cmd   ·   icacls
hola.txt
```

AVANZADO

Inventario de controladores, credenciales guardadas, ejecución con otro usuario y permisos (ACL) de un archivo. icacls muestra (I) heredado, (F) control total, (RX) lectura y ejecución; se otorgan con /grant y se quitan con /remove o /deny.

DÓNDE VERLO DANIEL.bat, taller baterías 1

```
curl wttr.in/cucuta   ·   rundll32 user32.dll,UpdatePerUserSystemParameters   ·
mshta "app.hta"
```

AVANZADO

Descargar por HTTP (curl viene incluido en Windows 10/11), refrescar la configuración visual tras cambiar el registro, y ejecutar aplicaciones HTA.

DÓNDE VERLO JORGE.bat, broma.bat

```
taskschd.msc   ·   start rundll32 user32.dll MessageBeep
```

BASE

Abrir el Programador de tareas y reproducir el sonido de notificación del sistema (truco del segundo ejercicio del taller).

DÓNDE VERLO taller baterías 1

C

Red: comandos de los talleres

Los comandos de auditoría y configuración de red que aparecen en el Taller No 4 y se combinan con los .bat de diagnóstico.

12
SINTAXIS

```
netstat -a   ·   -o   ·   -ano   ·   -b
```

BASE

Todas las conexiones y puertos con su estado; -o añade el PID dueño, -n evita resolver nombres (más rápido y fácil de filtrar) y -b muestra el ejecutable asociado, lo que exige permisos de administrador.

DÓNDE VERLO JORGE.bat, DANIEL.bat, WinSysToolkit-ELKIN.bat

```
nslookup www.google.com
```

BASE

Consulta el DNS y devuelve la IP del dominio. Un mismo dominio puede devolver IPs distintas porque hay varios servidores.

DÓNDE VERLO taller redes

```
nslookup -type=mx www.ufps.edu.co   ·   -type=ptr
```

AVANZADO

Consulta los registros MX (servidores de correo del dominio) y los PTR (resolución inversa hostname ↔ IP).

DÓNDE VERLO taller redes

```
arp -a   ·   arp -a -v   ·   arp -s IP MAC
```

AVANZADO

Muestra la tabla ARP (qué IP local corresponde a qué MAC, con tipo DINÁMICO o ESTÁTICO) y permite fijar una entrada manual.

DÓNDE VERLO taller redes

```
net view   ·   net view \\NOMBREPC /ALL
```

BASE

Lista los equipos visibles en la red local y los recursos compartidos de uno concreto. Falla con error 6118 cuando el descubrimiento está bloqueado o la red es PÚBLICA.

DÓNDE VERLO taller redes

```
tracert www.google.com   ·   pathping www.google.com
```

BASE

tracert muestra cada salto (hop) de la ruta con su IP y tiempo; pathping combina ping y tracert y además mide la pérdida de paquetes por nodo (más lento).

DÓNDE VERLO taller redes

```
netsh interface show interface
```

BASE

Estado de todas las interfaces de red: nombre, si está conectada y su modo. netsh sin argumentos entra a su propio entorno interactivo (netsh>).

DÓNDE VERLO JORGE.bat, WinSysToolkit-ELKIN.bat

```
netsh interface ipv4 set address "Ethernet" source=static address=192.168.1.6
mask=255.255.255.0 gateway=192.168.1.1
```

AVANZADO

Configura IP estática. Respeta los espacios antes de los signos =: netsh los exige. Para volver a DHCP se usa el mismo comando terminando en «dhcp».

DÓNDE VERLO JORGE.bat, EjemplosBatch.bat

```
netsh interface ipv4 set dnsservers "Ethernet" static address=8.8.8.8 primary
```

AVANZADO

Fija el DNS primario; con add dnsservers ... index=2 se agrega uno secundario y con «dhcp» vuelve a automático.

DÓNDE VERLO taller redes

```
netsh advfirewall firewall add rule name="Puerto TCP 443" dir=in action=allow protocol=TCP localport=443
```

AVANZADO

Crea una regla de firewall para abrir un puerto: action=allow para abrir, dir=in entrada (dir=out salida). Con action=block se cierra y con delete rule name=... se elimina.

DÓNDE VERLO taller redes, ROYNER.bat

> ! Si creas otra regla con el mismo nombre, Windows añade una nueva en vez de reemplazarla: borra la anterior primero.

```
netsh wlan show wlanreport
```

AVANZADO

Genera un informe del WiFi en HTML, XML y CAB: consulta NCSI (conectividad), NDIS (controlador), EAP (autenticación) y WCM (gestión de conexiones).

DÓNDE VERLO taller redes

```
nmap -sVC -O 192.168.20.72
```

PELIGRO

Herramienta externa de escaneo: -sV detecta servicios y versiones, -sC ejecuta scripts básicos y -O identifica el sistema operativo. Se usa para verificar que las reglas de firewall funcionan.

DÓNDE VERLO taller redes

D

Chuletas de referencia rápida

Las tablas que conviene tener a mano: escapes de caracteres, códigos de color, comparadores y la lista mínima que exige el taller.

4
TABLAS

CARACTERES QUE HAY QUE ESCAPAR CON ^

CARÁCTER	SIGNIFICADO NORMAL	CÓMO IMPRIMIRLO O USARLO LITERAL
`^`	Escape: neutraliza el carácter siguiente	^^
`&`	Separador de comandos	^&
`|`	Tubería	^|
`< >`	Redirección de entrada y salida	^< ^>
`( )`	Agrupación de bloques	^(^)
`%`	Inicio de variable	%%
`->`	Flecha leída como redirección	-^>
`"`	Comillas: se imprimen tal cual	—

CÓDIGOS DE COLOR DE LA CONSOLA

CÓDIGO	FONDO	TEXTO
`0A`	Negro	Verde claro (clásico Matrix)
`1F`	Azul	Blanco brillante
`4F`	Rojo	Blanco brillante
`06`	Negro	Amarillo
`2F / 4E / 5A`	Verde / Rojo / Púrpura	Blanco / Amarillo / Verde claro
`(sin argumentos)`	Restablece negro	Restablece blanco

COMPARADORES Y VERIFICACIONES

SINTAXIS	QUÉ COMPRUEBA
`if "%a%"=="%b%"`	Igualdad de texto (siempre entre comillas)
`if /i "a"=="A"`	Igualdad sin distinguir mayúsculas
`if not ...`	Niega cualquier condición
`EQU / NEQ`	Igual / distinto (numérico)
`LSS / LEQ`	Menor / menor o igual
`GTR / GEQ`	Mayor / mayor o igual
`if exist "ruta"`	Existe archivo o carpeta
`if exist "ruta\"`	Existe y es una carpeta
`if errorlevel N`	errorlevel mayor o igual que N
`if %errorlevel%==0`	El comando anterior terminó bien

LISTA MÍNIMA DEL TALLER NO 5

COMANDO	PARA QUÉ
`ECHO / @ECHO OFF`	Imprimir texto / apagar el eco
`REM o ::`	Comentarios
`MKDIR / RMDIR`	Crear y borrar directorios
`DEL / COPY`	Borrar y copiar archivos
`TITLE / COLOR`	Título y colores de la ventana
`PAUSE / PAUSE > NUL`	Pausa con y sin mensaje
`ECHO.`	Línea en blanco
`SET / SET /a / SET /p`	Declarar, calcular y leer del teclado
`EXIT 0 / 1`	Cerrar con salida correcta o incorrecta